

Open-Access Quant Research & Trading Engine - AI Coding Agent PRD (Deep Spec)

1. Project Context & Objective

The objective is to engineer a highly modular, headless quantitative research and paper-trading engine. This system autonomously ingests open-access market data (Equities, Macro), engineers stationary predictive features, trains machine learning models (XGBoost/LSTM) to forecast risk-adjusted forward returns, and validates these models via strict, transaction-cost-aware vectorized backtesting.

The ultimate goal is to produce a production-ready research pipeline that mitigates common quant pitfalls (Look-ahead bias, Survivorship bias, Overfitting) while operating strictly within a \$0 data budget constraints.

2. Tech Stack & Versioning Dependencies

The coding agent must strictly adhere to these specific library versions to guarantee compatibility between vectorized backtesting and data ingestion layers.

- **Language:** `python` ≥ 3.11 (Strict PEP 484 Type Hinting required on all functions)
- **Data Storage:** `duckdb` $\geq 0.10.0$, `pyarrow` (Parquet integration)
- **Data Ingestion:** `yfinance` $\geq 0.2.37$, `pandas-datareader` , `requests-cache`
- **Quantitative & Math Ops:** `pandas` ≥ 2.0 , `numpy` ≥ 1.24 , `scipy`
- **Feature Engineering:** `ta` (Technical Analysis library), `statsmodels` (for stationarity tests)
- **Machine Learning:** `xgboost` ≥ 2.0 , `torch` ≥ 2.2 (PyTorch with CUDA/MPS support), `optuna` (Hyperparameter optimization)
- **Backtesting & Simulation:** `vectorbt` $\geq 0.24.0$ (Do *not* use `vectorbtpro` as it is paid)
- **Serialization & Ops:** `joblib` , `loguru` (for advanced asynchronous logging)

3. Strict Anti-Goals & Architectural Boundaries

Do not waste compute or tokens generating code for the following:

1. **NO Live Broker Integration / API Keys:** The execution module (`paper_trader.py`) must only simulate fills based on closing prices and track a virtual SQLite/DuckDB portfolio ledger.
2. **NO Look-Ahead Bias:** Features calculated at time t must only use data available up to time t . The target variable (e.g., $t + 5$ returns) must be explicitly shifted backwards during training and dropped during inference.
3. **NO In-Memory Data Swamps:** Do not hold the entire universe's history in pandas DataFrames. Query strictly what is needed from DuckDB using SQL, process in chunks if

necessary, and write back to Parquet.

4. **NO Arbitrary Train/Test Splits:** Do not use

`sklearn.model_selection.train_test_split` with `shuffle=True`. Time-series data must strictly use Purged Walk-Forward Cross-Validation (Embargo) to prevent data leakage.

5. **NO Unstationary Features:** Do not feed raw prices directly to XGBoost. All price data must be transformed into stationary series (returns, z-scores, or fractionally differentiated features).

4. Deep Architecture & File Structure

The agent must generate the following strict, modular directory tree.

```

/quant-engine
├── /config
│   ├── settings.yaml      # Universe definition, Hyperparameters, Filep
│   └── logging.yaml       # Loguru configurations
├── /src
│   ├── /data
│   │   ├── yf_client.py   # Asynchronous yfinance ingestion with retry/
│   │   ├── macro_client.py # FRED data ingestion (e.g., 10Y Treasury, CP
│   │   └── duckdb_repo.py  # DuckDB connection pool and CRUD operations
│   ├── /features
│   │   ├── technical.py   # MACD, RSI, ATR, Bollinger Bands logic
│   │   ├── statistical.py # Z-Scores, Rolling Volatility, Skewness
│   │   └── labelling.py    # Target variable generation (e.g., Forward R
│   ├── /models
│   │   ├── xgb_trainer.py # XGBoost model class with Optuna tuning
│   │   ├── lstm_trainer.py # PyTorch LSTM network with Early Stopping
│   │   └── walk_forward.py # Purged K-Fold Cross Validation logic
│   ├── /execution
│   │   ├── vectorbt_engine.py # Vectorized backtesting, Slippage, Fees
│   │   └── portfolio_state.py # Paper trading ledger and state management
│   └── /utils
│       ├── math_utils.py   # Helper functions for stationarity/scaling
│       └── metrics.py      # Sharpe, Sortino, Calmar, Max Drawdown calcula
├── /storage
│   ├── /db
│   │   └── quant_data.duckdb # Main database file
│   └── /artifacts
│       ├── /models          # .pkl (XGBoost) and .pt (PyTorch) binaries
│       └── /scalers          # scikit-learn standard scalers fitted on Tra
├── orchestrator.py         # Main CLI entry point (Argparse enabled)
├── requirements.txt
└── README.md
  
```

5. Exhaustive Data Schema & Payloads

The system relies on DuckDB for rapid analytical processing. Ensure exact typing.

5.1 DuckDB Relational Schema

```

-- Core Price Data (Resolution: Daily)
CREATE TABLE IF NOT EXISTS ohlcv_daily (
    ticker VARCHAR,
    date DATE,
    open DOUBLE,
    high DOUBLE,
    low DOUBLE,
    close DOUBLE,
    adj_close DOUBLE,
    volume BIGINT,
    PRIMARY KEY (ticker, date)
);

-- Macro Economic Indicators (Resolution: Daily/Monthly interpolated)
CREATE TABLE IF NOT EXISTS macro_data (
    date DATE PRIMARY KEY,
    us_10y_yield DOUBLE,          -- DGS10 from FRED
    us_2y_yield DOUBLE,          -- DGS2 from FRED
    vix_close DOUBLE             -- ^VIX from yfinance
);

-- Feature Store (Pre-computed to save training time)
CREATE TABLE IF NOT EXISTS feature_store (
    ticker VARCHAR,
    date DATE,
    -- Technicals
    ret_1d DOUBLE,               -- 1-day log return
    volatility_20d DOUBLE,       -- 20-day rolling standard deviation of ret_1d
    rsi_14 DOUBLE,              -- 14-day Relative Strength Index (0-100)
    macd_hist DOUBLE,           -- MACD Histogram (12, 26, 9)
    atr_14_pct DOUBLE,          -- Average True Range normalized by close price
    -- Cross-Sectional (Z-Scores within the sector/universe for the given date)
    z_score_ret_1m DOUBLE,       -- Z-score of 1-month return vs universe
    -- Target Variable
    target_fwd_ret_5d DOUBLE,    -- Forward 5-day continuously compounded return
    target_fwd_ret_5d_bin INT,   -- Binary classification: 1 if > 0, else 0
    PRIMARY KEY (ticker, date)
);

-- Backtest Execution Ledger (Paper Trader State)
CREATE TABLE IF NOT EXISTS paper_trades (
    trade_id UUID PRIMARY KEY,
    ticker VARCHAR,
    signal_date DATE,
    execution_date DATE,         -- Usually signal_date + 1 (trading on next ope
    direction INT,              -- 1 (Long), -1 (Short)
    execution_price DOUBLE,
    position_size DOUBLE,       -- Fractional shares or exact quantity
    transaction_cost DOUBLE,    -- Simulated broker fees
    status VARCHAR              -- 'OPEN', 'CLOSED'
);

```

5.2 Python Data Contracts (Pydantic / TypedDict)

```

from pydantic import BaseModel
from typing import Literal

```

```

import datetime

class TradeSignal(BaseModel):
    ticker: str
    signal_date: datetime.date
    direction: Literal[1, 0, -1]
    confidence_score: float          # Probability output [0.0, 1.0] from model
    predicted_alpha: float           # Expected return magnitude
    stop_loss_pct: float             # e.g., 0.05 for 5%
    take_profit_pct: float           # e.g., 0.10 for 10%

class BacktestMetrics(BaseModel):
    model_id: str
    total_return_pct: float
    annualized_sharpe: float         # Assuming Risk Free Rate = 0.04
    max_drawdown_pct: float
    win_rate_pct: float
    total_trades: int
    profit_factor: float             # Gross Profit / Gross Loss

```

6. Strict Mathematical Formulations

The AI must implement feature engineering exactly as defined below to ensure mathematical soundness.

- **Log Returns:** Use continuously compounded returns for additive properties over time.

$$R_t = \ln \left(\frac{P_t}{P_{t-1}} \right)$$

- **Volatility (Annualized 20-day):**

$$Vol_{20,t} = \sqrt{252} \times \sqrt{\frac{1}{19} \sum_{i=0}^{19} (R_{t-i} - \bar{R}_{20})^2}$$

- **Target Variable Definition (Forward 5-Day Return):** To avoid look-ahead bias, the feature row at index t must align with the target calculated from future prices.

$$Target_t = \ln \left(\frac{Open_{t+6}}{Open_{t+1}} \right)$$

Implementation Note: We calculate the target using the *Open* of $t + 1$ to the *Open* of $t + 6$. This reflects the realistic execution constraint of receiving a signal at the close of day t and executing the trade at the open of day $t + 1$.

- **Z-Score Cross-Sectional Normalization:** For a given date t , for stock i in universe U :

$$Z_{i,t} = \frac{F_{i,t} - \mu_{U,t}}{\sigma_{U,t}}$$

Where F is a specific feature, μ is the cross-sectional mean, and σ is the cross-sectional standard deviation.

7. Step-by-Step Implementation Protocol

Phase 1: Robust Data Ingestion (`yf_client.py` & `macro_client.py`)

1. **Universe Definition:** Parse `settings.yaml` for a list of tickers (e.g., `['BBCA.JK', 'BMRI.JK', 'AAPL', 'MSFT']`).
2. **Rate-Limit Handling:** Implement `requests_cache` with a SQLite backend to prevent spamming Yahoo Finance. Implement exponential backoff for `HTTP 429 Too Many Requests` errors.
3. **Data Validation:** Ensure fetched data has no missing index dates (fill missing non-trading days where necessary, or align strictly to trading calendars). Forward-fill `macro_data` to match equity trading days.
4. **Storage:** Upsert raw pandas DataFrames into the DuckDB `ohlcv_daily` and `macro_data` tables.

Phase 2: Feature Engineering & Target Labelling (`features/`)

1. **Vectorized Calculation:** Query `ohlcv_daily` .Group by `ticker` and apply rolling pandas operations (`.rolling(window=20).mean()` , etc.) and `ta` library calculations.
2. **Target Generation:** Shift the price column negatively (`shift(-5)`) to create the forward target. **CRITICAL:** The last 5 rows of the dataset will have `NaN` for the target. These must be explicitly dropped during training, but kept during live inference.
3. **Cross-Sectional Merging:** Merge macro features (Yield curve spread, VIX) onto the equity dataframe matching by `date` .
4. **Scaling/Normalization:** Fit `StandardScaler` on the training set *only*. Save the scaler to `/artifacts/scalers/` via `joblib` . Transform the validation/test sets using the fitted scaler. Write the finalized dataframe to DuckDB `feature_store` .

Phase 3: Model Architecture & Training (`xgb_trainer.py`)

1. **Purged Walk-Forward Split:** Implement a custom cross-validation iterator.
 - *Train:* Jan 2018 - Dec 2020
 - *Embargo Period:* 5 days (matches target horizon to prevent label leakage).
 - *Validation:* Jan 2021 - Dec 2021
2. **Optuna Tuning:** Define an objective function optimizing for the highest *Sharpe Ratio* on the validation set, NOT just standard log-loss or RMSE.
 - Search space: `max_depth`

, learning_rate

0.01, 0.1

, subsample

0.6, 0.9

.

3. **Training & Serialization:** Train the final model on the optimal parameters. Serialize the .pk1 file with a timestamped naming convention (e.g., xgb_model_20260506_1530.pk1).

Phase 4: Vectorized Backtesting (vectorbt_engine.py)

1. **Signal Generation:** Load the latest .pk1 model. Generate continuous probabilities for the Test set dataset.
2. **Thresholding:** Convert probabilities into discrete signals.
 - If $P(Y = 1) > 0.65 \Rightarrow \text{Signal} = 1$ (Long)
 - If $P(Y = 1) < 0.35 \Rightarrow \text{Signal} = -1$ (Short)
 - Else $\Rightarrow \text{Signal} = 0$ (Neutral/Hold)
3. **Vectorbt Execution:** Pass the arrays to vbt.Portfolio.from_signals() .
 - **Parameters:** init_cash=10000 , fees=0.0015 (0.15% to simulate IDX/standard retail fees), slippage=0.0005 (5 bps).
4. **Reporting:** Generate a tearsheet outputting to the console via loguru . Must include: Win Rate, Sharpe, Max Drawdown, and Total Return.

Phase 5: Paper Trading Execution (paper_trader.py)

1. **State Management:** On daily cron execution, the orchestrator pulls the *current* day's close.
2. **Inference:** The model generates $t + 1$ signals for the universe.
3. **Ledger Update:** The paper_trader.py reads the signals, checks available mock capital in the SQLite/DuckDB paper_trades state, and writes the hypothetical trades intended for the next day's open.
4. **Mark-to-Market:** The ledger updates the equity curve based on the actual open prices of $t + 1$ on the subsequent run.

8. Error Handling & Edge Case Protocols

The coding agent must wrap critical pipeline junctions in defensive programming constructs.

- **Data Holes:** If a ticker has $> 5\%$ missing data points in the lookback window, drop the ticker from the current training epoch. Do not interpolate excessively.

- **Collinearity:** XGBoost handles collinearity well, but standard scaling must check for zero-variance features (e.g., a stock that hasn't traded/halted). Add a near-zero variance filter before model ingestion.
- **Outliers:** Cap/Floor extreme returns (e.g., returns $> 300\%$ in a day due to data vendor